

This image shows a full page of white paper with horizontal dotted lines, typical of primary school writing paper. The lines are evenly spaced and run across the width of the page. There are no margins, text, or other markings on the paper.

(Ký tên và ghi rõ họ tên)

Trần Hoàng Long

[illegible]

Thành viên hội đồng
(Ký tên và ghi rõ họ tên)

LỜI CẢM ƠN

Kính thưa quý Thầy Cô trong Hội đồng chấm bảo vệ Đồ án cơ sở ngành, Kính thưa thầy Khấu Văn Nhựt – Giảng viên hướng dẫn,

Để hoàn thành đồ án cơ sở ngành này, bên cạnh sự nỗ lực của bản thân, em đã nhận được sự quan tâm, động viên và hướng dẫn tận tình từ quý Thầy Cô và bạn bè.

Lời đầu tiên, em xin gửi lời tri ân sâu sắc nhất đến thầy Khấu Văn Nhựt – người đã trực tiếp hướng dẫn và đồng hành cùng em trong suốt quá trình thực hiện đề tài. Thầy không chỉ định hướng phương pháp nghiên cứu, giải đáp những thắc mắc về mặt kỹ thuật mà còn giúp em rèn luyện tư duy giải quyết vấn đề một cách khoa học. Sự tận tâm và những lời góp ý quý báu của Thầy chính là kim chỉ nam giúp em hoàn thiện đồ án "Xây dựng trò chơi Rắn Săn Mồi" một cách tốt nhất.

Em cũng xin gửi lời cảm ơn chân thành đến quý Thầy Cô trong Hội đồng chấm thi. Em xin cảm ơn quý Thầy Cô đã dành thời gian quý báu để đọc, đánh giá và đưa ra những nhận xét, phản biện thiết thực. Những ý kiến đóng góp của Hội đồng không chỉ giúp em nhìn nhận ra những thiếu sót trong đồ án lần này mà còn là những bài học kinh nghiệm vô giá cho chặng đường phát triển nghề nghiệp sau này.

Mặc dù đã rất cố gắng, nhưng do kiến thức và kinh nghiệm thực tế còn hạn chế nên đồ án khó tránh khỏi những thiếu sót. Em rất mong nhận được sự thông cảm và chỉ bảo thêm từ quý Thầy Cô.

Em xin kính chúc quý Thầy Cô luôn dồi dào sức khỏe, hạnh phúc và gặt hái được nhiều thành công hơn nữa trong sự nghiệp trồng người cao quý.

Em xin trân trọng cảm ơn!

MỤC LỤC

DANH MỤC HÌNH ẢNH	7
DANH MỤC BẢNG BIỂU	7
TÓM TẮT ĐỒ ÁN CƠ SỞ NGÀNH	9
MỞ ĐẦU	10
CHƯƠNG 1: TỔNG QUAN.....	14
1.1 Đặt vấn đề.....	14
1.1.1 Lịch sử hình thành	14
1.1.2 Cơ chế hoạt động	15
1.1.3 Tổng quan các nghiên cứu liên quan	15
1.2 Mục tiêu và Nhiệm vụ nghiên cứu	17
1.2.1 Mục tiêu tổng quát:	17
1.2.2 Nhiệm vụ cụ thể	17
1.3 Phạm vi và đối tượng nghiên cứu.....	19
1.4 Công cụ và môi trường phát triển.....	20
CHƯƠNG 2: NGHIÊN CỨU LÝ THUYẾT	22
2.1 Kiến trúc Unity và C#	22
2.1.1 Component-Based Architecture.....	22
2.1.2 Vòng đời GameObject	22
2.1.3 Hệ thống Rendering 2D	22
2.2 Các Mẫu thiết kế (Design Patterns) áp dụng.....	23
2.2.1 Singleton Pattern.....	23
2.2.2 Observer Pattern	24
2.2.3 ScriptableObject	25
2.2.4 Object Pooling Pattern.....	25
2.3 Cơ sở thuật toán.....	26

2.3.1 BFS (Breadth-First Search)	26
2.3.2 Flood Fill.....	26
2.3.3 Heuristic Functions	26
2.3.4 Grid-based Collision Detection	27
2.4 Toán học ứng dụng trong Game.....	27
2.4.1 Hàm lượng giác trong Animation.....	27
2.4.2 Vector Mathematics.....	28
2.4.3 Interpolation Techniques	28
CHƯƠNG 3: HIỆN THỰC HÓA NGHIÊN CỨU	30
3.1 Thiết kế hệ thống (System Design)	30
3.1 Sơ đồ Use Case	30
3.1.1 Thiết kế Sơ đồ lớp (Class Diagram)	31
3.2 Các thuật toán cốt lõi (Core Algorithms)	32
3.2.1 Hệ thống lưới tọa độ và Va chạm (Grid System)	32
3.2.2 Cơ chế di chuyển và Input Buffering.....	33
3.2.3 Thuật toán sinh thức ăn thông minh	33
3.3 Các kỹ thuật lập trình nâng cao (Advanced Techniques).....	35
3.3.1 Trí tuệ nhân tạo (AI) - Chiến thuật kếp	35
3.3.2 Hệ thống Đa ngôn ngữ (Localization)	37
3.3.3 Quản lý dữ liệu và Tối ưu hóa	38
3.4 Trải nghiệm người dùng và Công cụ hỗ trợ (UX & Tools)	38
3.4.1 Hiệu ứng thị giác (Visual Polish)	38
3.4.2 Toán học trong Hiệu ứng thị giác (Procedural Animation).....	38
3.4.3 Camera thích ứng.....	39
3.4.4 Ứng dụng Toán học trong Hiệu ứng chuyển động	40
CHƯƠNG 4: KẾT QUẢ NGHIÊN CỨU	42

4.1	Kết quả hiện thực	42
4.2	Kịch bản kiểm thử (Test Cases)	42
4.3	Đánh giá hiệu năng (Performance).....	44
4.4	Giao diện và trải nghiệm người dùng.....	44
CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN.....		49
5.1	Kết luận	49
5.2	Hạn chế.....	49
5.3	Hướng phát triển trong tương lai.....	49
DANH MỤC TÀI LIỆU THAM KHẢO		50

DANH MỤC HÌNH ẢNH

<i>Hình 1: Sơ đồ Use Case tổng quát của hệ thống.....</i>	<i>30</i>
<i>Hình 2: Sơ đồ Class Diagram.....</i>	<i>31</i>
<i>Hình 3: Hệ thống lưới tọa độ (Grid System) được trực quan hóa trong Unity.....</i>	<i>33</i>
<i>Hình 4: Các loại vật phẩm (Thường, Hiếm, Cực hiếm).</i>	<i>35</i>
<i>Hình 5: Mô phỏng nguyên lý hoạt động của thuật toán tìm kiếm theo chiều rộng. .</i>	<i>35</i>
<i>Hình 6: Giao diện Menu Chính.....</i>	<i>45</i>
<i>Hình 7: Giao diện cài đặt</i>	<i>45</i>
<i>Hình 8: Giao diện bảng xếp hạng.....</i>	<i>46</i>
<i>Hình 9: Màn hình Gameplay</i>	<i>47</i>
<i>Hình 10: Màn hình Game Over.....</i>	<i>47</i>

DANH MỤC BẢNG BIỂU

Bảng 1: So sánh và lựa chọn hướng tiếp cận	17
Bảng 2: Danh sách các mục kiểm thử.....	43

TÓM TẮT ĐỒ ÁN CƠ SỞ NGÀNH

Việc phát triển trò chơi điện tử không chỉ đòi hỏi sự sáng tạo về nội dung mà còn yêu cầu sự vững chắc về kỹ thuật lập trình. Đồ án tập trung nghiên cứu và xây dựng lại trò chơi kinh điển "Rắn săn mồi" (Snake Game) – một biểu tượng của dòng game Arcade, nhưng được nâng cấp toàn diện về mặt công nghệ và trải nghiệm người dùng.

Vấn đề nghiên cứu chính là khắc phục sự đơn điệu của lối chơi truyền thống và sự hạn chế về đồ họa trên các phiên bản cũ. Để giải quyết vấn đề này, đồ án sử dụng Unity Engine làm nền tảng phát triển chính, kết hợp với ngôn ngữ lập trình C# và tư duy lập trình hướng đối tượng (OOP). Đặc biệt, đồ án đi sâu vào việc cài đặt các thuật toán tìm đường (Pathfinding Algorithms) như BFS và Flood Fill trên hệ thống lưới (Grid System) để xây dựng hệ thống Trí tuệ nhân tạo (AI) có khả năng tự động săn mồi và tránh vật cản thông minh.

Kết quả đạt được là một sản phẩm game hoàn chỉnh hoạt động ổn định trên nền tảng PC với FPS đạt 60 khung hình/giây. Game tích hợp thành công hai chế độ chơi: Chơi đơn (Single Player) và Đấu với máy (PvE), cùng hệ thống tính điểm Combo và các hiệu ứng đồ họa (VFX) hiện đại theo phong cách Neon. Hệ thống AI được xây dựng với hai chiến thuật: Săn mồi (Hunting Mode) khi có đường đi an toàn và Sinh tồn (Survival Mode) khi bị bao vây. Ngoài ra, sản phẩm còn hỗ trợ đa ngôn ngữ (Tiếng Việt/Tiếng Anh), lưu trữ dữ liệu người chơi và tùy chỉnh giao diện theo sở thích.

Đồ án không chỉ dừng lại ở một sản phẩm giải trí mà còn minh chứng cho khả năng ứng dụng các thuật toán cơ sở vào thực tế phát triển phần mềm. Qua quá trình thực hiện, sinh viên đã áp dụng thành công các Design Patterns như Singleton, Observer, Object Pooling để tối ưu hóa hiệu năng và dễ dàng bảo trì code. Sản phẩm có thể mở rộng thêm các tính năng như chế độ Multiplayer Online, Machine Learning cho AI và phát hành trên nền tảng Mobile trong tương lai.

MỞ ĐẦU

1. Lý do chọn đề tài

Game không đơn thuần là công cụ giải trí mà đã trở thành một ngành công nghiệp tỷ đô, đồng thời là môi trường tuyệt vời để rèn luyện tư duy logic và kỹ năng giải quyết vấn đề. Theo báo cáo của Newzoo (2024), thị trường game toàn cầu đạt giá trị 184 tỷ USD với 3.2 tỷ người chơi trên toàn thế giới. Việc tham gia vào lĩnh vực này không chỉ mang lại cơ hội nghề nghiệp mà còn giúp phát triển kỹ năng lập trình toàn diện.

Là sinh viên chuyên ngành Công nghệ thông tin, việc tiếp cận quy trình phát triển game giúp củng cố kiến thức về lập trình hướng đối tượng, cấu trúc dữ liệu và giải thuật một cách trực quan nhất. Thay vì chỉ học lý thuyết trên giấy, việc xây dựng một game hoàn chỉnh cho phép áp dụng đồng thời nhiều kiến thức: từ thuật toán tìm đường, quản lý bộ nhớ, xử lý sự kiện, đến thiết kế giao diện và tối ưu hóa hiệu năng.

Trò chơi Rắn săn mồi (Snake Game) là một tượng đài của dòng game arcade, ra đời từ những năm 1970 và đạt đỉnh cao phổ biến khi được tích hợp vào điện thoại Nokia vào cuối thập niên 1990. Tuy nhiên, các phiên bản cũ thường thiếu sự đa dạng về lối chơi, đồ họa đơn điệu (chỉ có màu đen trắng) và không có đối thủ AI thông minh. Xuất phát từ niềm đam mê lập trình và mong muốn tạo ra một sản phẩm mang dấu ấn cá nhân, em quyết định chọn đề tài này để phát triển một phiên bản nâng cấp toàn diện.

Đề tài này cũng phù hợp với chương trình Đồ án Cơ sở ngành vì có độ phức tạp vừa phải: đủ thách thức để áp dụng nhiều kiến thức chuyên môn nhưng không quá khó để hoàn thành trong thời gian quy định. Hơn nữa, kết quả của đồ án là một sản phẩm cụ thể, có thể demo trực quan, dễ dàng đánh giá và có khả năng mở rộng trong tương lai.

2. Mục tiêu nghiên cứu Đồ án đặt ra các mục tiêu cụ thể sau:

Đồ án đặt ra các mục tiêu cụ thể sau:

Về kiến thức: Nắm vững kiến trúc Component-based của Unity, hiểu rõ vòng đời đối tượng (GameObject Lifecycle) và cách quản lý bộ nhớ hiệu quả để

tránh rò rỉ bộ nhớ (Memory Leak) và phân mảnh bộ nhớ (Memory Fragmentation). Đồng thời, củng cố kiến thức về lập trình hướng đối tượng (OOP) thông qua việc thiết kế các lớp có tính kế thừa, đóng gói và đa hình.

Về kỹ thuật: Cài đặt thành công các thuật toán tìm đường (Pathfinding Algorithms) như BFS và Flood Fill cho hệ thống AI. Xây dựng hệ thống lưới tọa độ (Grid System) với độ chính xác tuyệt đối, không phụ thuộc vào Physics Engine của Unity. Áp dụng các Design Patterns phổ biến trong ngành như Singleton, Observer, Object Pooling để code dễ bảo trì và mở rộng.

Về sản phẩm: Tạo ra một sản phẩm hoàn thiện với giao diện đẹp mắt theo phong cách Neon Style, hỗ trợ đa ngôn ngữ (Tiếng Việt/Tiếng Anh), có hệ thống lưu trữ điểm cao (High Score) và tích hợp công cụ gỡ lỗi (Debug Tools) để tiện cho việc phát triển và kiểm thử. Game phải đạt hiệu năng ổn định 60 FPS trên máy tính cấu hình trung bình.

Về kỹ năng mềm: Rèn luyện khả năng tự học, tìm kiếm tài liệu kỹ thuật, đọc hiểu API Documentation và giải quyết vấn đề một cách độc lập. Phát triển kỹ năng quản lý thời gian, lập kế hoạch và trình bày kết quả nghiên cứu một cách khoa học.

3. Đối tượng và phạm vi nghiên cứu

Đối tượng nghiên cứu: Đồ án tập trung vào ba nhóm đối tượng chính như:

Các thuật toán đồ thị (Graph Theory) áp dụng trong game: BFS, Flood Fill, Heuristic Search

Cấu trúc dữ liệu phù hợp cho game 2D: Queue, Stack, HashSet, Dictionary

Kiến trúc phần mềm theo mô hình Component-Based và Event-Driven Architecture

Phạm vi nghiên cứu:

Nền tảng: PC (Windows), không triển khai trên Mobile hoặc Web

Chế độ chơi: Single Player và Player vs AI (PvE), không có Multiplayer qua mạng

Công nghệ: Unity 2025, C#, không sử dụng thư viện bên ngoài ngoài các package có sẵn của Unity

Giới hạn: Không đi sâu vào thiết kế đồ họa 3D, Network Programming, hoặc các kỹ thuật Machine Learning phức tạp

4. Phương pháp nghiên cứu

Đồ án áp dụng phương pháp nghiên cứu thực nghiệm (Experimental Research) kết hợp với phương pháp phát triển phần mềm theo mô hình Agile. Quy trình được chia thành các giai đoạn:

Giai đoạn 1 - Nghiên cứu tài liệu (1 tuần): Tìm hiểu Unity Engine, đọc tài liệu về các thuật toán, xem các tutorial và phân tích code của các dự án mã nguồn mở tương tự.

Giai đoạn 2 - Thiết kế hệ thống (1 tuần): Vẽ Use Case Diagram, Class Diagram, lên kế hoạch về kiến trúc code và lựa chọn Design Patterns phù hợp.

Giai đoạn 3 - Phát triển Core (2 tuần): Xây dựng các module cốt lõi: Grid System, Snake Movement, Collision Detection, Food Spawning.

Giai đoạn 4 - Phát triển AI (2 tuần): Cài đặt thuật toán BFS, Flood Fill và logic chuyển đổi giữa hai chế độ Hunting/Survival.

Giai đoạn 5 - Polish và Testing (1 tuần): Thêm hiệu ứng hình ảnh, âm thanh, localization, và tiến hành kiểm thử toàn diện.

Giai đoạn 6 - Tài liệu hóa (1 tuần): Viết báo cáo, chuẩn bị slide thuyết trình và demo video.

5. Đóng góp của đồ án

Đồ án không chỉ là bài tập lớn mà còn mang lại những đóng góp thiết thực:

Về mặt học thuật: Đồ án minh chứng cách áp dụng các thuật toán cơ sở (Data Structures & Algorithms) vào một bài toán thực tế. Qua đó, giúp sinh viên hiểu sâu hơn về độ phức tạp thuật toán (Big O Notation) và cách tối ưu hóa hiệu năng.

Về mặt kỹ thuật: Cung cấp một codebase mẫu được tổ chức tốt, tuân thủ các nguyên tắc SOLID và Clean Code, có thể tham khảo cho các dự án Unity tương lai.

Về mặt ứng dụng: Sản phẩm có thể được phát triển thành một game thương mại hoàn chỉnh hoặc làm nền tảng cho các bài học về lập trình game cho người mới bắt đầu.

CHƯƠNG 1: TỔNG QUAN

1.1 Đặt vấn đề

Ngành công nghiệp trò chơi điện tử (Game Industry) đang phát triển với tốc độ vũ bão, trở thành một trong những lĩnh vực công nghệ mũi nhọn. Bên cạnh các tựa game bom tấn (AAA) với đồ họa siêu thực và gameplay phức tạp, dòng game Arcade cổ điển vẫn giữ được sức hút riêng nhờ lối chơi đơn giản nhưng đầy tính thử thách. Điều này chứng tỏ rằng, một game không nhất thiết phải có đồ họa 3D đẹp mắt mới thu hút người chơi, mà quan trọng hơn là gameplay gây nghiện và cảm giác thành tựu khi vượt qua thử thách.

1.1.1 Lịch sử hình thành

Trò chơi Snake (Rắn săn mồi) có lịch sử phát triển lâu dài và đầy thú vị:

Giai đoạn đầu (1976-1977): Game Snake đầu tiên được phát triển bởi Gremlin Industries dưới tên "Blockade" cho máy Arcade. Trong phiên bản này, hai người chơi điều khiển hai đường line chuyển động, mục tiêu là chặn đường đối phương.

Giai đoạn phát triển (1978-1980): Các biến thể như "Hustle" và "Worm" xuất hiện trên các nền tảng máy tính cá nhân đời đầu như TRS-80 và Apple II. Đây là lúc gameplay cơ bản được định hình: một con rắn ăn mồi và tránh va chạm với chính mình.

Giai đoạn bùng nổ (1997-2000): Snake thực sự trở thành hiện tượng toàn cầu khi Nokia tích hợp sẵn game này vào dòng điện thoại Nokia 6110 năm 1997. Với màn hình đơn sắc và điều khiển đơn giản bằng phím bấm, Snake đã chinh phục hàng trăm triệu người dùng trên toàn thế giới. Đến năm 1998, game đã được cài đặt trên hơn 350 triệu thiết bị Nokia.

Thời kỳ hiện đại (2000-nay): Snake được remake trên vô số nền tảng với đồ họa và gameplay được cải tiến. Các phiên bản nổi bật như Slither.io (2016) với chế độ Multiplayer Online, Snake vs Block với cơ chế phá gạch, hay Google Snake doodle với nhiều theme đồ họa khác nhau.

1.1.2 Cơ chế hoạt động

Về mặt kỹ thuật, Snake hoạt động dựa trên cơ chế lưới (Grid-Based System). Không giống như các game platformer hay RPG sử dụng hệ tọa độ liên tục (Continuous Space), Snake hoạt động trên một mảng 2D rời rạc (Discrete Grid)

Cấu trúc dữ liệu cơ bản: Rắn được biểu diễn bằng một danh sách liên kết (Linked List) hoặc Queue, trong đó mỗi node chứa tọa độ của một đốt rắn. Khi rắn di chuyển, một node mới được thêm vào đầu (head) và node cuối (tail) bị loại bỏ. Khi ăn mồi, node cuối không bị xóa, do đó rắn tăng chiều dài.

Vòng lặp game (Game Loop): Game chạy theo vòng lặp vô hạn với ba giai đoạn chính:

Input: Đọc phím nhấn từ người chơi

Update: Tính toán vị trí mới, kiểm tra va chạm

Render: Vẽ lại màn hình với trạng thái mới

Kiểm tra va chạm (Collision Detection): Có hai loại va chạm chính:

Va chạm với tường:

$\text{if (headX} < 0 \parallel \text{headX} \geq \text{gridWidth} \parallel \text{headY} < 0 \parallel \text{headY} \geq \text{gridHeight})$

Va chạm với thân rắn: Sử dụng HashSet để kiểm tra xem tọa độ đầu rắn có trùng với bất kỳ đốt nào không

Sinh thức ăn (Food Spawning): Thức ăn được sinh ra tại một ô trống ngẫu nhiên. Thuật toán cơ bản là: lấy danh sách tất cả các ô trống, chọn ngẫu nhiên một ô trong đó. Tuy nhiên, để tránh thức ăn sinh quá gần rắn (dẫn đến game quá dễ), cần áp dụng các heuristic như Manhattan Distance.

1.1.3 Tổng quan các nghiên cứu liên quan

Để định hướng cho đề án, em đã khảo sát và phân tích các sản phẩm và nghiên cứu liên quan:

Nokia Snake (1997) - Phiên bản gốc:

Ưu điểm: Đơn giản, dễ chơi, không yêu cầu cấu hình cao

Nhược điểm: Đồ họa đơn sắc, không có AI, không có các powerup đặc biệt

Bài học: Gameplay đơn giản nhưng gây nghiện là yếu tố quan trọng nhất

Slither.io (2016) - Multiplayer Online:

Ưu điểm: Chế độ Multiplayer hấp dẫn, đồ họa màu sắc, có tính cạnh tranh cao

Nhược điểm: Phụ thuộc vào kết nối mạng, có lag nếu ping cao

Bài học: Multiplayer tạo ra sự cạnh tranh nhưng cần xử lý tốt Network Latency

Snake AI với thuật toán Hamiltonian Cycle (2018):

Nghiên cứu của Christer Kaitila: Xây dựng AI hoàn hảo cho Snake bằng cách tạo một chu trình Hamilton trên lưới. AI sẽ đi theo chu trình này và đảm bảo không bao giờ tự va chạm.

Ưu điểm: AI không bao giờ chết

Nhược điểm: Di chuyển không tự nhiên, mất nhiều thời gian để hoàn thành game

Bài học: AI quá hoàn hảo không tạo ra trải nghiệm thú vị

Snake với Deep Reinforcement Learning (2020):

Nghiên cứu của các nhóm tại Stanford, MIT: Sử dụng Deep Q-Network (DQN) để huấn luyện AI tự học chơi Snake

Kết quả: Sau hàng nghìn episode huấn luyện, AI có thể đạt điểm số cao hơn người chơi trung bình

Hạn chế: Yêu cầu GPU mạnh và thời gian huấn luyện lâu, không phù hợp cho đồ án cơ sở ngành

Bài học: Machine Learning mạnh nhưng cần tài nguyên lớn, không nên áp dụng cho dự án nhỏ

Snake với thuật toán A (2019):*

Ưu điểm: Tìm đường tối ưu nhanh hơn BFS nhờ hàm heuristic

Nhược điểm: Phức tạp hơn về mặt cài đặt, cần điều chỉnh trọng số heuristic

Bài học: A* phù hợp cho game có nhiều vật cản phức tạp

Bảng 1: So sánh và lựa chọn hướng tiếp cận

Tiêu chí	BFS	A*	Hamiltonian	Deep RL
Độ phức tạp cài đặt	Đơn giản	Trung bình	Phức tạp	Rất phức tạp
Hiệu năng	Tốt	Rất tốt	Trung bình	Tốt
Tính tự nhiên	Cao	Cao	Thấp	Cao
Yêu cầu tài nguyên	Thấp	Thấp	Thấp	Rất cao

1.2 Mục tiêu và Nhiệm vụ nghiên cứu

1.2.1 Mục tiêu tổng quát:

Xây dựng hoàn thiện một sản phẩm game 2D trên nền tảng PC, đáp ứng đầy đủ các tiêu chuẩn của một phần mềm giải trí chuyên nghiệp: ổn định về mặt kỹ thuật (không có bug nghiêm trọng, không crash), mượt mà về hiệu năng (60 FPS ổn định), giao diện thân thiện (UI/UX trực quan, dễ sử dụng), và có tính thử thách phù hợp (AI đủ thông minh để tạo ra trải nghiệm thú vị).

Đồ án hướng tới việc áp dụng các kiến thức đã học trong chương trình đào tạo vào thực tế một cách hiệu quả nhất, bao gồm:

Cấu trúc dữ liệu và giải thuật (Data Structures & Algorithms), lập trình hướng đối tượng (Object-Oriented Programming), kiến trúc phần mềm và Design Patterns, tối ưu hóa hiệu năng (Performance Optimization)

1.2.2 Nhiệm vụ cụ thể

Để đạt được mục tiêu trên, đồ án cần thực hiện các nhiệm vụ kỹ thuật sau:

Nhiệm vụ 1 - Nghiên cứu Unity Engine:

Làm chủ quy trình quản lý Scene, Prefab và vòng đời của GameObject, hiểu rõ các hàm trong MonoBehaviour: Awake(), Start(), Update(), FixedUpdate(), LateUpdate(), nắm vững cơ chế Event System và UnityEvent, thành thạo công cụ Inspector, Hierarchy, Project View và Console

Nhiệm vụ 2 - Xây dựng Core Gameplay:

Thiết kế hệ thống di chuyển dựa trên lưới (Grid Movement) với độ chính xác tuyệt đối, thay vì sử dụng Physics2D(BoxCollider, Rigidbody) thường gây sai số do floating point, đồ án tự xây dựng hệ thống va chạm riêng dựa trên tọa độ số nguyên, cài đặt Input Buffering để tránh "nuốt phím" khi người chơi thao tác nhanh, xử lý logic ăn mồi, tăng trưởng và tính điểm

Nhiệm vụ 3 - Phát triển Trí tuệ nhân tạo:

Cài đặt thuật toán tìm đường BFS (Breadth-First Search) để AI tìm đường ngắn nhất đến thức ăn, áp dụng thuật toán Flood Fill để AI đánh giá độ an toàn của các hướng đi, xây dựng logic chuyển đổi giữa hai chế độ: Hunting (săn mồi) và Survival (sinh tồn)

Cân bằng độ khó: AI phải đủ thông minh để tạo thử thách nhưng không quá mạnh đến mức người chơi không thể thắng

Nhiệm vụ 4 - Tối ưu hóa hệ thống:

Áp dụng Singleton Pattern cho các Manager (GameManager, GridManager, AudioManager), sử dụng Object Pooling cho âm thanh và visual effects để giảm GC Allocation, áp dụng Observer Pattern (UnityEvent, C# Action) để giảm coupling giữa các module, sử dụng ScriptableObject để tách data ra khỏi logic code.

Nhiệm vụ 5 - Nâng cao trải nghiệm người dùng:

Tích hợp hệ thống đa ngôn ngữ (Localization) hỗ trợ Tiếng Việt và Tiếng Anh, xây dựng hệ thống lưu trữ dữ liệu (PlayerPrefs hoặc JSON) cho High Score và Settings.

Thiết kế các hiệu ứng hình ảnh (VFX): Glow, Outline, Pulse, Particle Effects

Tích hợp âm thanh: Background Music, Sound Effects với Volume Control

Nhiệm vụ 6 - Kiểm thử và Debug:

Viết Test Cases cho các chức năng chính, sử dụng Unity Profiler để phân tích hiệu năng

Xây dựng Debug Tools: Grid Visualization, Path Visualization cho AI

Xử lý các Edge Cases: rắn quá dài, không còn ô trống, input conflict

1.3 Phạm vi và đối tượng nghiên cứu

Phạm vi không gian:

Nền tảng: Đồ án được triển khai trên môi trường Windows (PC) với hệ điều hành Windows 10/11

Độ phân giải: Hỗ trợ các độ phân giải phổ biến từ 1280x720 (HD) đến 1920x1080 (Full HD)

Không triển khai: Mobile (Android/iOS), WebGL, Console (PlayStation, Xbox)

Phạm vi kỹ thuật:

Ngôn ngữ: C# (phiên bản 9.0 trở lên) với .NET Standard 2.1

Engine: Unity 2023 LTS (Long Term Support)

Không sử dụng: Thư viện Machine Learning (ML-Agents), Network Library (Photon, Mirror), External Plugins

Tập trung: 2D Rendering, Sprite Animation, Grid System, Pathfinding Algorithms

Phạm vi chức năng:

Có: Single Player, Player vs AI, Pause/Resume, Settings, Leaderboard

Không có: Multiplayer Online, Level Editor, User-generated Content, In-app Purchase

Đối tượng nghiên cứu:

Thuật toán đồ thị (Graph Theory):

BFS (Breadth-First Search) với độ phức tạp $O(V+E)$, Flood Fill với độ phức tạp $O(N^2)$ trong trường hợp xấu nhất.

Heuristic Functions: Manhattan Distance, Euclidean Distance

Cấu trúc dữ liệu:

Queue (FIFO) cho BFS

HashSet cho collision detection $O(1)$

Dictionary cho localization và data mapping

List cho snake segments

Kiến trúc phần mềm:

Component-Based Architecture

Event-Driven Architecture

Design Patterns: Singleton, Observer, Object Pool, ScriptableObject

1.4 Công cụ và môi trường phát triển

Đồ án sử dụng bộ công cụ tiêu chuẩn trong ngành phát triển game:

Unity Engine (Phiên bản 2025 LTS): Unity là một trong những game engine phổ biến nhất thế giới, được sử dụng bởi hơn 50% các game mobile và 60% VR/AR content. Lý do chọn Unity:

Đa nền tảng: Hỗ trợ build cho 25+ platforms khác nhau

Cộng đồng lớn: Hơn 2.8 triệu developers và kho tài liệu phong phú

Asset Store: Marketplace với 74,000+ assets miễn phí và trả phí

Miễn phí cho sinh viên: Unity Student Plan cung cấp đầy đủ tính năng

Các tính năng chính được sử dụng:

2D Renderer: Sprite Renderer, Sprite Atlas, 2D Tilemap

Animation System: Animator Controller, Animation Clips

UI System: Canvas, Event System, Text Mesh Pro

Audio: Audio Source, Audio Mixer

Scripting: MonoBehaviour, Coroutine, Unity Events

Visual Studio 2022 (IDE): Visual Studio là môi trường phát triển tích hợp được Microsoft phát triển, tích hợp sâu với Unity, hỗ trợ:

IntelliSense: Auto-complete thông minh cho Unity API

Debugger: Breakpoints, Watch Window, Call Stack

Unity Integration: Attach to Unity, Edit and Continue

Code Analysis: Warnings, Suggestions, Refactoring Tools

Visual Paradigm (UML Tool): Visual Paradigm là công cụ vẽ biểu đồ UML chuyên nghiệp, hỗ trợ:

Use Case Diagram, Class Diagram với Auto Layout, Sequence Diagram, Activity Diagram, Export sang PNG, PDF, SVG.

Adobe Photoshop CC 2024: Photoshop được sử dụng cho:

Thiết kế UI elements: buttons, panels, icons

Chỉnh sửa sprites: snake segments, food items

Tạo hiệu ứng: glow, blur, shadows

Optimize cho game: Export as PNG-8 với transparency

Công cụ bổ sung:

Git/GitHub: Version control, collaboration, repository management

Audacity: Chỉnh sửa âm thanh và tạo sound effects

Unity Profiler: Phân tích hiệu năng, memory usage

Snipping Tool: Chụp màn hình cho tài liệu

CHƯƠNG 2: NGHIÊN CỨU LÝ THUYẾT

2.1 Kiến trúc Unity và C#

2.1.1 Component-Based Architecture

Unity là một trong những Game Engine phổ biến nhất thế giới hiện nay, cung cấp môi trường phát triển tích hợp (IDE) mạnh mẽ và khả năng xuất bản đa nền tảng [5]. Khác với lập trình hướng đối tượng truyền thống, Unity sử dụng kiến trúc Hướng thành phần (Component-Based Architecture). Trong đó, GameObject đóng vai trò là vật chứa (Container), và các hành vi (Behavior) được định nghĩa thông qua các Component gắn vào nó. Kiến trúc này cho phép các nhà phát triển lắp ghép các tính năng một cách linh hoạt mà không cần tạo ra các lớp kế thừa phức tạp [4].

2.1.2 Vòng đời GameObject

Hiểu rõ vòng đời của MonoBehaviour là yếu tố then chốt để kiểm soát luồng game. Theo tài liệu kỹ thuật của Unity Technologies [7], các phương thức quan trọng bao gồm:

Awake(): Được gọi đầu tiên ngay khi script được khởi tạo. Đây là nơi lý tưởng để thiết lập các tham chiếu Singleton hoặc khởi tạo các biến nội bộ.

Start(): Được gọi trước frame đầu tiên (ngay sau Awake), dùng để thiết lập trạng thái ban đầu của game.

Update(): Được gọi liên tục mỗi frame. Đây là nơi xử lý tín hiệu đầu vào (Input) từ người chơi và cập nhật logic di chuyển thời gian thực.

OnDestroy(): Được gọi khi đối tượng bị hủy, dùng để dọn dẹp bộ nhớ, hủy đăng ký sự kiện (Event Unsubscription) để tránh rò rỉ bộ nhớ.

2.1.3 Hệ thống Rendering 2D

Unity 2D sử dụng SpriteRenderer để hiển thị hình ảnh. Để quản lý thứ tự hiển thị (Draw Order) của các đối tượng chồng lấn nhau, đồ án sử dụng hệ thống Sorting Layer và Order in Layer được cấu hình như sau:

Background (Order -10): Hình nền phía sau cùng.

Grid (Order 0): Lưới bàn cờ.

Food (Order 5): Thức ăn (nằm trên lưới).

Snake (Order 10): Rắn (nằm trên thức ăn).

Effects (Order 20): Các hiệu ứng nổ, phát sáng.

UI (Canvas): Lớp giao diện nằm trên cùng.

2.2 Các Mẫu thiết kế (Design Patterns) áp dụng

Việc áp dụng các mẫu thiết kế giúp mã nguồn trở nên "sạch", dễ bảo trì và mở rộng. Theo Robert Nystrom trong cuốn *Game Programming Patterns* [6], đồ án sử dụng các mẫu thiết kế chính sau:

2.2.1 Singleton Pattern

Singleton đảm bảo một lớp (Class) chỉ có duy nhất một thể hiện (Instance) tồn tại trong suốt vòng đời ứng dụng và cung cấp một điểm truy cập toàn cục [3].

Ứng dụng trong đồ án: Singleton được áp dụng cho các lớp quản lý trung tâm nhằm đảm bảo tính nhất quán của dữ liệu:

GameManager: Quản lý trạng thái game (Start, Pause, Game Over) và điểm số.

GridManager: Quản lý hệ tọa độ lưới và xử lý va chạm.

AudioManager: Quản lý phát nhạc và âm thanh.

```
public class Singleton<T> : MonoBehaviour where T : MonoBehaviour {
    private static T instance;
    public static T Instance {
        get {
            if (instance == null) {
                instance = FindObjectOfType<T>();
                if (instance == null) {
                    GameObject singleton = new GameObject($"[Singleton]
{typeof(T).Name}");
                    instance = singleton.AddComponent<T>();
                    DontDestroyOnLoad(singleton);
                }
            }
            return instance;
        }
    }
}
```

2.2.2 Observer Pattern

Mẫu thiết kế này cho phép một đối tượng (Subject) thông báo cho các đối tượng khác (Observers) về sự thay đổi trạng thái của nó mà không cần tạo liên kết chặt chẽ (Loose Coupling) [6].

Ứng dụng trong đồ án: Đồ án sử dụng UnityEvent để triển khai Observer Pattern, giúp tách biệt Logic và Giao diện (UI):

Khi rắn ăn mồi, SnakeController phát sự kiện. GameManager nghe sự kiện để cộng điểm, AudioManager nghe để phát âm thanh.

Khi rắn chết, sự kiện OnGameOver được kích hoạt để UIManager hiển thị bảng điểm mà không cần Rắn phải tham chiếu trực tiếp đến UI.

```
public class GameManager : Singleton<GameManager>
{
    public UnityEvent<int> OnScoreChanged = new UnityEvent<int>();
    public UnityEvent OnGameOver = new UnityEvent();

    public void AddScore(int points)
    {
        score += points;
        OnScoreChanged.Invoke(score);
    }
}

// UI Script
public class ScoreUI : MonoBehaviour
{
    void Start()
    {
        GameManager.Instance.OnScoreChanged.AddListener(UpdateScore);
    }

    void UpdateScore(int newScore)
    {
        scoreText.text = $"Score: {newScore}";
    }
}
```


2.2.3 ScriptableObject

ScriptableObject là một tính năng đặc biệt của Unity dùng để lưu trữ dữ liệu độc lập với Class Instance. Đồ án sử dụng nó cho ColorPalette để quản lý các bộ màu sắc (Theme) cho game. Điều này giúp Designer có thể tinh chỉnh màu sắc ngay trong Editor mà không cần can thiệp vào code, đồng thời dữ liệu được bảo toàn khi khởi động lại.

2.2.4 Object Pooling Pattern

Thay vì hủy (Destroy) và khởi tạo lại (Instantiate) đối tượng liên tục gây áp lực lên bộ dọn rác (Garbage Collector), Object Pooling tái sử dụng các đối tượng đang ở trạng thái "ngủ" [6].

Ứng dụng trong đồ án: Được áp dụng trong AudioManager để quản lý các âm thanh hiệu ứng (SFX). Một danh sách (List) chứa sẵn 10-20 AudioSource được tạo ra từ đầu. Khi cần phát tiếng động, hệ thống lấy một nguồn âm đang rảnh để sử dụng, giúp game hoạt động mượt mà, không bị giật (lag).

Implementation cho Audio System:

```
public class AudioManager : Singleton<AudioManager>{
    private List<AudioSource> sfxPool = new List<AudioSource>();
    void Awake(){
        // Tạo pool với 10 AudioSources
        for (int i = 0; i < 10; i++){
            GameObject poolObj = new GameObject($"SFXPool_{i}");
            poolObj.transform.SetParent(transform);
            AudioSource source = poolObj.AddComponent<AudioSource>();
            source.playOnAwake = false;
            sfxPool.Add(source);
        }
    }
    public void PlaySFX(AudioClip clip){
        AudioSource source = sfxPool.Find(s => ! s.isPlaying);
        if (source != null){
            source.clip = clip;
            source.Play();
        }
    }
}
```

2.3 Cơ sở thuật toán

2.3.1 BFS (Breadth-First Search)

BFS là thuật toán tìm kiếm theo chiều rộng, duyệt đồ thị bắt đầu từ đỉnh gốc và mở rộng ra tất cả các đỉnh lân cận theo từng lớp độ sâu.

Nguyên lý: Sử dụng cấu trúc dữ liệu Hàng đợi (Queue) để lưu trữ các đỉnh cần duyệt và Tập hợp (HashSet) để đánh dấu các đỉnh đã thăm [1].

Độ phức tạp: $O(V + E)$ với V là số ô lưới (1200 ô) và E là số cạnh nối (tối đa 4 cạnh/ô). Trong bối cảnh lưới 40x30, thuật toán thực thi gần như tức thì ($O(1)$ trong thực tế).

Lý do lựa chọn: Theo tài liệu về giải thuật [2][9], trên đồ thị không trọng số (Unweighted Graph) như lưới game Snake, BFS luôn đảm bảo tìm ra đường đi ngắn nhất (Shortest Path) với chi phí tính toán thấp hơn A^* (do không cần tính hàm Heuristic khoảng cách).

2.3.2 Flood Fill

Flood Fill là thuật toán dùng để xác định các vùng liên thông trong một mảng đa chiều.

Ứng dụng trong AI: Khi AI bị bao vây và không tìm được đường đến thức ăn, thuật toán Flood Fill được kích hoạt để đếm số lượng ô trống (Free Space) ở 4 hướng di chuyển. AI sẽ chọn hướng dẫn đến vùng không gian rộng nhất để "câu giờ", chờ cơ hội thoát ra.

Hiệu quả: Giúp AI thông minh hơn, biết sinh tồn thay vì đâm đầu vào tường ngay lập tức khi mất đường đi.

2.3.3 Heuristic Functions

Khoảng cách Manhattan: Đồ án sử dụng công thức Manhattan ($|x1 - x2| + |y1 - y2|$) thay vì khoảng cách Euclid để ước lượng chi phí di chuyển, vì rắn chỉ có thể di chuyển theo 4 hướng vuông góc trên lưới [2].

Xử lý va chạm với HashSet: Thay vì duyệt toàn bộ danh sách rắn (độ phức tạp $O(N)$) để kiểm tra va chạm, đồ án sử dụng `HashSet<Vector2Int>`. Cấu

trúc dữ liệu này cho phép kiểm tra sự tồn tại của một phần tử chỉ với độ phức tạp $O(1)$, giúp tối ưu hóa hiệu năng cực đại khi con rắn dài ra.

2.3.4 Grid-based Collision Detection

Thay vì sử dụng Unity Physics (BoxCollider2D, Rigidbody2D), đồ án tự xây dựng collision system dựa trên HashSet.

Nguyên lý:

```
HashSet<Vector2Int> occupiedCells = new HashSet<Vector2Int>();

// Thêm vật cản
occupiedCells.Add(new Vector2Int(5, 10)); // O(1)

// Kiểm tra va chạm
bool isBlocked = occupiedCells.Contains(headPosition); // O(1)
```

So sánh với List:

```
List<Vector2Int> occupiedCells = new List<Vector2Int>();

// Kiểm tra va chạm
bool isBlocked = occupiedCells.Contains(headPosition); // O(N)
```

2.4 Toán học ứng dụng trong Game

Phát triển game không chỉ là viết logic mà còn áp dụng nhiều kiến thức toán học để tạo ra các chuyển động mượt mà và vật lý chân thực. Đồ án sử dụng các kiến thức sau:

2.4.1 Hàm lượng giác trong Animation

Hàm Sin cho chuyển động nhấp nhô: $y(t) = y_0 + A \times \sin(\omega \times t)$

Trong đó:

y_0 : Vị trí ban đầu

A : Biên độ dao động

ω : Tần số góc (omega) $= 2\pi f$

t : Thời gian

```
float y = startY + amplitude * Mathf.Sin(Time.time * frequency);
transform.position = new Vector3(x, y, z);
```

Ví dụ trong đồ án: Với amplitude = 0.1f và frequency = 2f, viên thức ăn sẽ di chuyển lên xuống trong khoảng ± 0.1 đơn vị với tốc độ 2 chu kỳ/giây.

Hàm Cos (Cosine): Tương tự hàm Sin nhưng lệch pha $\pi/2$, thường được kết hợp để tạo chuyển động xoay hoặc quỹ đạo tròn.

```
float angle = Mathf.Cos(Time.time * rotateSpeed) * maxAngle;  
transform.rotation = Quaternion.Euler(0, 0, angle);
```

2.4.2 Vector Mathematics

Vector cơ bản:

Vector2Int(1, 0) → Right

Vector2Int(-1, 0) → Left

Vector2Int(0, 1) → Up

Vector2Int(0, -1) → Down

Dot Product (Tích vô hướng):

$$A \cdot B = |A| \times |B| \times \cos(\theta)$$

Ứng dụng: Kiểm tra hướng đi hợp lệ

```
// Không cho phép đi ngược chiều  
if (Vector2Int.Dot(newDirection, currentDirection) == -1)  
{  
    return; // Không cho phép quay đầu  
}
```

Magnitude (Độ dài vector):

```
float distance = (posA - posB).magnitude;
```

2.4.3 Interpolation Techniques

Để tránh việc các đối tượng di chuyển giật cục (teleport) từ điểm A sang điểm B, đồ án sử dụng các kỹ thuật nội suy để làm mượt chuyển động.

Nội suy tuyến tính (Lerp - Linear Interpolation): Tìm một điểm nằm giữa A và B dựa trên tham số t chạy từ 0 đến 1.

Công thức: $Result = A + (B - A) \times t$

Ứng dụng: Làm mượt thanh máu, thanh thời gian hoặc chuyển màu rắn

```
// Di chuyển mượt từ A đến B  
Vector3 smoothPos = Vector3.Lerp(currentPos, targetPos, Time.deltaTime *  
speed);
```

SmoothDamp: Là một biến thể nâng cao của nội suy, mô phỏng chuyển động vật lý có gia tốc và giảm tốc (như lò xo). Kỹ thuật này được dùng cho CameraController để camera bám theo rắn một cách tự nhiên, không bị cứng nhắc.

```
float smoothTime = 0.3f;  
float velocity = 0f;  
float smoothY=Mathf.SmoothDamp(currentY,targetY,ref velocity,smoothTime);
```

CHƯƠNG 3: HIỆN THỰC HÓA NGHIÊN CỨU

3.1 Thiết kế hệ thống (System Design)

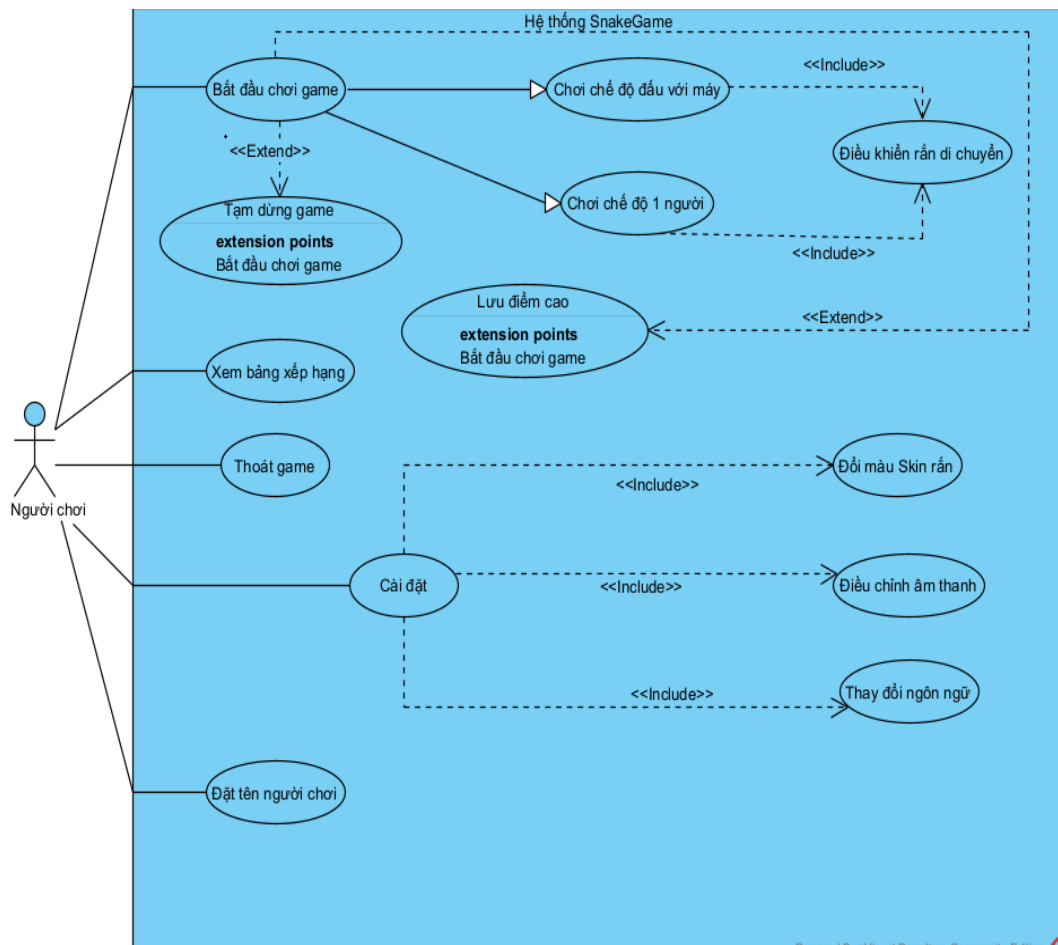
3.1 Sơ đồ Use Case

Hệ thống được thiết kế phục vụ người chơi với các nhóm chức năng chính:

Chức năng chơi game (Core Gameplay): Bao gồm chế độ chơi đơn truyền thống (Single Player) và chế độ thách đấu với máy (Vs AI). Trong đó, hệ thống AI đóng vai trò là một tác nhân tự động có khả năng ra quyết định.

Chức năng hệ thống: Cài đặt âm thanh (Sound/Music), thay đổi ngôn ngữ (Localization), và tùy chỉnh giao diện (Theme Color).

Chức năng dữ liệu: Hệ thống tự động lưu trữ điểm cao (High Score) và trạng thái cài đặt của người dùng xuống bộ nhớ máy.



Hình 1: Sơ đồ Use Case tổng quát của hệ thống.

3.1.1 Thiết kế Sơ đồ lớp (Class Diagram)

Hệ thống tuân thủ kiến trúc hướng đối tượng (OOP) với sự phân chia trách nhiệm rõ ràng giữa các lớp:

Tầng Quản lý (Managers):

GameManager: Áp dụng mẫu Singleton, chịu trách nhiệm điều phối vòng đời của game (Start, Pause, Game Over).

GridManager: Quản lý hệ thống tọa độ và trạng thái các ô trên bản đồ.

Tầng Thực thể (Entities):

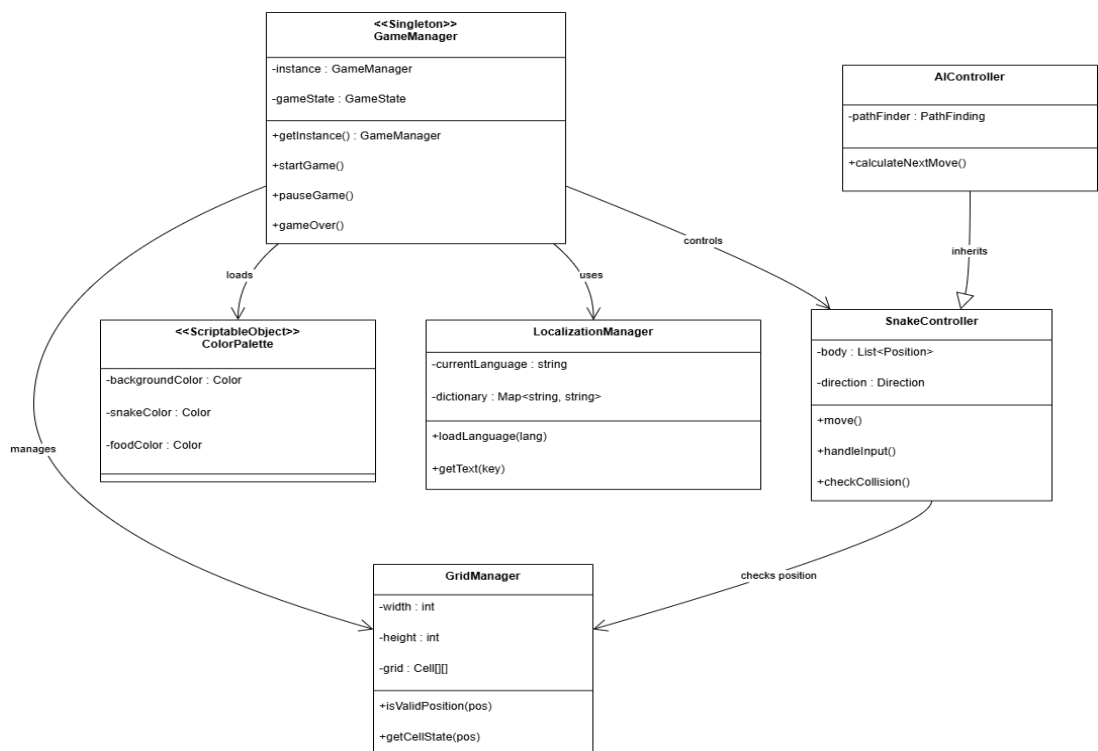
SnakeController: Lớp cơ sở quản lý di chuyển và input.

AIController: Kế thừa từ SnakeController, bổ sung thuật toán tìm đường.

Tầng Dữ liệu (Data):

ColorPalette (ScriptableObject): Lưu trữ cấu hình màu sắc.

LocalizationManager: Quản lý từ điển đa ngôn ngữ.



Hình 2: Sơ đồ Class Diagram

3.2 Các thuật toán cốt lõi (Core Algorithms)

3.2.1 Hệ thống lưới tọa độ và Va chạm (Grid System)

Thiết kế Grid:

Kích thước: 40×30 cells

Mỗi cell: 1×1 Unity unit

Tổng diện tích: $40 \times 30 = 1200$ cells

Chuyển đổi tọa độ:

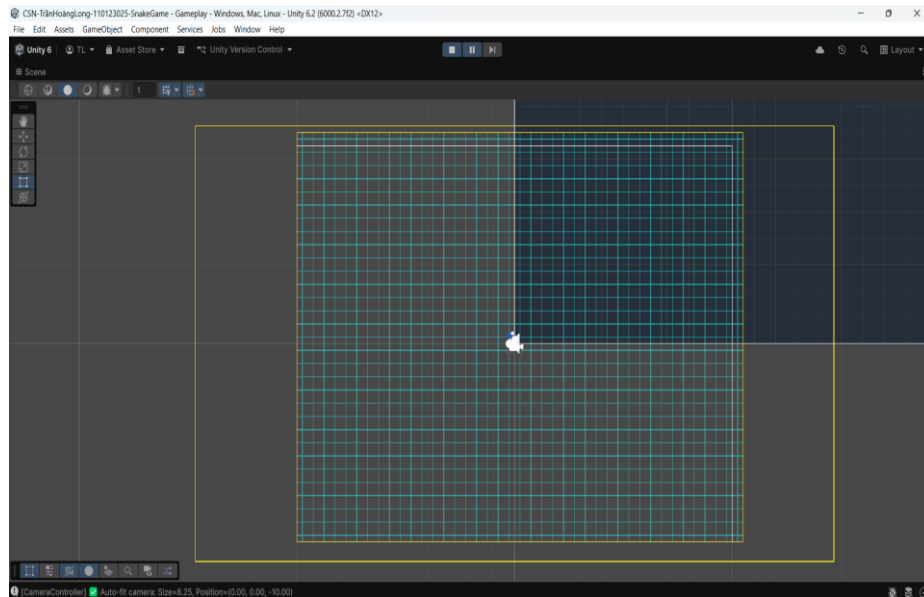
```
public class AIController : MonoBehaviour{
    [SerializeField] private AIStrategy strategy = AIStrategy.Balanced;
    [SerializeField] private float decisionInterval = 0.1f;
    private SnakeController snake;
    private AIPathfinding pathfinding;
    public enum AIStrategy { Aggressive, Defensive, Balanced }
    void Update(){
        if (Time.time >= nextDecisionTime){
            MakeDecision();
            nextDecisionTime = Time.time + decisionInterval;
        }
    }
    void MakeDecision(){
        Vector2Int currentPos = snake.GetHeadPosition();
        List<Vector2Int> obstacles = GetAllObstacles();
        // Mode 1: Hunting - Tìm đường đến food
        List<Vector2Int> path = pathfinding.FindPath(currentPos, foodPos, obstacles);
        if (path != null && path.Count > 0){
            Vector2Int nextMove = path[0] - currentPos;
            snake.SetDirection(nextMove);}
        else{
            Vector2Int safeDir = FindSafestDirection(currentPos, obstacles);
            snake.SetDirection(safeDir);}
        Vector2Int FindSafestDirection(Vector2Int pos, List<Vector2Int> obstacles){
            Vector2Int[] directions={ Vector2Int.up, Vector2Int.down, Vector2Int.left,
            Vector2Int.right};
            int maxSpace = 0;
            Vector2Int bestDir = snake.GetCurrentDirection();
            foreach (var dir in directions){
                int space = FloodFillCount(pos + dir, obstacles);
                if (space > maxSpace){
                    maxSpace = space;
                    bestDir = dir;
                }
            }
            return bestDir;
        }
    }
}
```


Tối ưu hóa:

HashSet thay vì List: $O(1)$ vs $O(N)$ cho Contains()

Vector2Int thay vì Vector2: tiết kiệm bộ nhớ, so sánh nhanh hơn

Không sử dụng Physics2D: tránh overhead của collision callbacks



Hình 3: Hệ thống lưới tọa độ (Grid System) được trực quan hóa trong Unity.

3.2.2 Cơ chế di chuyển và Input Buffering

Để giải quyết vấn đề "nuốt phím" (Input Lag) thường gặp khi người chơi thao tác quá nhanh, hệ thống sử dụng kỹ thuật **Input Buffering**. Biến nextDirection hoạt động như một bộ đệm, lưu trữ phím bấm và chỉ áp dụng hướng đi mới khi chu kỳ vật lý tiếp theo bắt đầu.

3.2.3 Thuật toán sinh thức ăn thông minh

Thức ăn không sinh hoàn toàn ngẫu nhiên mà tuân theo các quy tắc:

Rules:

Không sinh trên ô đã chiếm (rắn, tường)

Cách đầu rắn ít nhất minDistance cells

Phân bố xác suất: 70% Normal, 25% Rare, 5% Epic

```

public class FoodSpawner : Singleton<FoodSpawner>
{
    [SerializeField] private int minDistanceFromSnakes = 3;
    [SerializeField] private float commonChance = 0.70f;
    [SerializeField] private float rareChance = 0.25f;
    // epicChance = 0.05f (còn lại)
    public void SpawnFood()
    {
        List<Vector2Int> validPositions = GetValidSpawnPositions();
        if (validPositions.Count == 0) return;
        Vector2Int spawnPos = validPositions[Random.Range(0,
validPositions.Count)];
        FoodType type = DetermineFoodType();
        GameObject food = Instantiate(GetFoodPrefab(type));
        food.transform.position = GridManager.Instance.GridToWorld(spawnPos);
    }
    List<Vector2Int> GetValidSpawnPositions()
    {
        List<Vector2Int> valid = new List<Vector2Int>();
        for (int x = 0; x < GridManager.Instance.GridWidth; x++){
            for (int y = 0; y < GridManager.Instance.GridHeight; y++){
                Vector2Int pos = new Vector2Int(x, y);
                if (GridManager.Instance.IsOccupied(pos)) continue;
                bool tooClose = false;
                foreach (var snake in allSnakes){
                    if (ManhattanDistance(pos, snake.GetHeadPosition()) <
minDistanceFromSnakes){
                        tooClose = true;
                        break;
                    }
                }

                if (!tooClose) valid.Add(pos);
            }
        }
        return valid;
    }
    FoodType DetermineFoodType(){
        float roll = Random.value;
        if (roll < commonChance) return FoodType.Common;
        if (roll < commonChance + rareChance) return FoodType.Rare;
        return FoodType.Epic;
    }
}

```



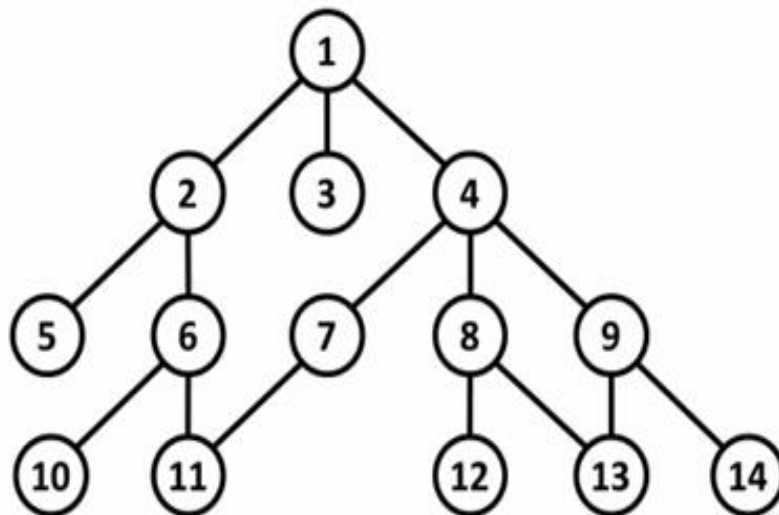
Hình 4: Các loại vật phẩm (Thường, Hiếm, Cực hiếm).

3.3 Các kỹ thuật lập trình nâng cao (Advanced Techniques)

3.3.1 Trí tuệ nhân tạo (AI) - Chiến thuật kếp

AI không chỉ tìm đường mà còn có tư duy chiến thuật:

Trạng thái Săn mồi (Hunting): Dùng BFS để tìm đường ngắn nhất đến thức ăn.



Hình 5: Mô phỏng nguyên lý hoạt động của thuật toán tìm kiếm theo chiều rộng.

```
public List<Vector2Int> FindPath(Vector2Int start, Vector2Int target,
List<Vector2Int> obstacles)
{
    Queue<Vector2Int> queue = new Queue<Vector2Int>();
    Dictionary<Vector2Int, Vector2Int> cameFrom = new Dictionary<Vector2Int,
Vector2Int>();
    queue.Enqueue(start);
    while (queue.Count > 0)
    {
        Vector2Int current = queue.Dequeue();
        if (current == target) return ReconstructPath(cameFrom, start, current);

        foreach (var neighbor in GetNeighbors(current))
        {
            if (!obstacles.Contains(neighbor) &&
!cameFrom.ContainsKey(neighbor))
            {
                queue.Enqueue(neighbor);
                cameFrom[neighbor] = current;
            }
        }
    }
    return null; // Không tìm thấy đường
}
```

Trạng thái Sinh tồn (Survival): Khi không tìm thấy đường (bị bao vây), AI kích hoạt thuật toán Flood Fill để tìm hướng đi dẫn đến vùng không gian rộng nhất, kéo dài thời gian sống sót.

```
private int CountFreeSpace(Vector2Int start, List<Vector2Int> obstacles){
    // Sử dụng BFS để đếm số lượng ô liên thông
    Queue<Vector2Int> q = new Queue<Vector2Int>();
    HashSet<Vector2Int> visited = new HashSet<Vector2Int>();
    q.Enqueue(start);
    visited.Add(start);
    int count = 0;
    while (q.Count > 0)
    {
        var current = q.Dequeue();
        count++;
        // ... logic duyệt lân cận ...
    }
    return count;
}
```

Đồng bộ tốc độ: Hàm SyncSpeedWithPlayer giúp AI tự động tăng tốc theo người chơi để duy trì độ khó.

3.3.2 Hệ thống Đa ngôn ngữ (Localization)

Hệ thống hỗ trợ Tiếng Việt/Tiếng Anh chuyển đổi tức thì.

Cấu trúc: Sử dụng Dictionary để tra cứu từ khóa.

Cập nhật động: Khi đổi ngôn ngữ, LocalizationManager phát lệnh RefreshAllLocalizedTexts, cập nhật toàn bộ UI mà không cần tải lại Scene.

```
public class LocalizationManager : Singleton<LocalizationManager>{
    public enum Language { Vietnamese, English }
    [SerializeField] private Language currentLanguage = Language.Vietnamese;
    private Dictionary<string, Dictionary<Language, string>> localizedStrings =
        new Dictionary<string, Dictionary<Language, string>>();
    void Awake(){
        LoadLocalizedStrings();
        LoadSavedLanguage();
    }
    void LoadLocalizedStrings()
    {
        AddString("menu. play", "Chơi", "Play");
        AddString("menu. settings", "Cài đặt", "Settings");
        AddString("game.score", "Điểm", "Score");
        AddString("game.game_over", "Kết thúc", "Game Over");
    }
    void AddString(string key, string vi, string en){
        localizedStrings[key] = new Dictionary<Language, string>{
            { Language.Vietnamese, vi },
            { Language.English, en }
        };
    }
    public string GetLocalizedString(string key) {
        if (localizedStrings.ContainsKey(key))
            return localizedStrings[key][currentLanguage];
        return key;
    }
    public void ChangeLanguage(Language newLang){
        currentLanguage = newLang;
        PlayerPrefs.SetString("Language", newLang.ToString());
        // Refresh all UI texts...
    }
}
```

3.3.3 Quản lý dữ liệu và Tối ưu hóa

JSON Serialization: Bảng xếp hạng được lưu dưới dạng chuỗi JSON mã hóa, bảo toàn dữ liệu phức tạp (Tên, Điểm, Ngày giờ).

Object Pooling: Hệ thống âm thanh sử dụng bể đối tượng (sfxPool) để tái sử dụng AudioSource, ngăn chặn tình trạng phân mảnh bộ nhớ (GC Allocation).

```
// Trích dẫn từ AudioManager.cs
private AudioSource GetAvailableSFXSource()
{
    foreach (AudioSource source in sfxPool)
    {
        if (!source.isPlaying) return source;
    }
    // Nếu hết pool, trả về null hoặc mở rộng pool
    return null;
}
```

3.4 Trải nghiệm người dùng và Công cụ hỗ trợ (UX & Tools)

3.4.1 Hiệu ứng thị giác (Visual Polish)

Các hiệu ứng như phát sáng (Glow), viền (Outline) và nhấp nháy (Pulse) được lập trình thủ công bằng thuật toán thay đổi SpriteRenderer và Color theo hàm Sin thời gian thực, tạo cảm giác "Game Juice" mượt mà.

```
// Trích dẫn từ OutlineEffect.cs
private void CreateOutline()
{
    // Tạo 4 bản sao dịch chuyển theo 4 hướng
    Vector3[] offsets = { Vector3.up, Vector3.down, Vector3.left, Vector3.right };
    foreach (var offset in offsets)
    {
        GameObject outline = new GameObject("Outline");
        // ... setup sprite renderer color & sorting order ...
    }
}
```

3.4.2 Toán học trong Hiệu ứng thị giác (Procedural Animation)

Thay vì sử dụng Animation Clip tốn bộ nhớ, các hiệu ứng chuyển động của thức ăn được tính toán bằng hàm lượng giác (Mathf.Sin) theo thời gian thực.

Hiệu ứng trôi nổi (Floating)

$$Y = Y_{start} = \sin(\text{Time.Speed}).\text{Amout}$$

Hiệu ứng nhịp đập (Pulse)

$$\text{Scale} = \text{Base} \cdot (1 + \sin(\text{Time.Speed}).\text{Amout})$$

3.4.3 Camera thích ứng

Để trò chơi hiển thị trọn vẹn trên mọi thiết bị (từ màn hình vuông 4:3 đến màn hình rộng 16:9), lớp CameraController sử dụng thuật toán tự động điều chỉnh kích thước vùng nhìn (orthographicSize).

Quy trình xử lý trong hàm FitCameraToGrid() như sau:

Lấy kích thước lưới: Truy xuất thông số GridWidth và GridHeight từ GridManager.

Tính toán tỷ lệ: So sánh tỷ lệ khung hình thiết bị (screenRatio) với tỷ lệ lưới (targetRatio).

Điều chỉnh Zoom:

Nếu màn hình hẹp hơn lưới: Tính toán dựa trên chiều rộng để tránh mất hình hai bên.

Nếu màn hình rộng hơn lưới: Tính toán dựa trên chiều cao.

Đoạn mã xử lý cốt lõi

```
private void FitCameraToGrid(){
    float targetRatio = (float)gridWidth / gridHeight;
    float screenRatio = (float)Screen.width / Screen.height;
    if (screenRatio >= targetRatio){
        cam.orthographicSize = (gridHeight / 2f) + padding;
    }
    else{
        float differenceInSize = targetRatio / screenRatio;
        cam.orthographicSize = (gridHeight / 2f * differenceInSize) + padding;
    }
}
```

3.4.4 Ứng dụng Toán học trong Hiệu ứng chuyển động

Thay vì sử dụng các Animation Clip có sẵn của Unity (vốn tiêu tốn dung lượng bộ nhớ và khó tùy biến), đồ án áp dụng các hàm lượng giác để tạo ra chuyển động mượt mà cho các đối tượng trong game.

Hiệu ứng trôi nổi (Floating Effect): Thức ăn trong game không đứng yên mà chuyển động nhấp nhô nhẹ nhàng để thu hút sự chú ý của người chơi. Vị trí theo trục Y của đối tượng được tính toán lại trong mỗi khung hình (Update) dựa trên hàm Sin.

Công thức toán học áp dụng:

$$Y_{new} = Y_{start} + \text{Amplitude} \cdot \sin(\text{Time} \cdot \text{Frequency})$$

Trong đó:

Y_{start}

Amplitude: Biên độ dao động

Frequency: Tần số dao động

Đoạn mã hiện thực trong FoodAnimation.cs

```
// Trích dẫn từ FoodAnimation.cs
private void Update()
{
    // Tạo chuyển động lên xuống mượt mà
    float yOffset = Mathf.Sin(Time.time * floatSpeed) * floatAmount;

    transform.position = startPosition + Vector3.up * yOffset;

    // Kết hợp xoay đều quanh trục Z
    if (rotate)
    {
        transform.Rotate(Vector3.forward, rotateSpeed * Time.deltaTime);
    }
}
```

Hiệu ứng Nhịp đập (Pulse Effect): Đối với các vật phẩm đặc biệt (Rare/Epic) hoặc các nút bấm UI, hiệu ứng co giãn (Scaling) được áp dụng để tạo cảm giác đối tượng đang "thở".

Đoạn mã hiện thực trong GlowPulse.cs

```
private void Update()
{
    // Tính toán hệ số scale từ 0 đến 1 dựa trên thời gian
    float t = (Mathf.Sin(Time.time * pulseSpeed) + 1f) * 0.5f;

    // Nội suy tuyến tính giữa kích thước nhỏ nhất và lớn nhất
    float scale = Mathf.Lerp(minScale, maxScale, t);

    transform.localScale = originalScale * scale;
}
```

CHƯƠNG 4: KẾT QUẢ NGHIỆM CỨU

4.1 Kết quả hiện thực

Sau 10 tuần làm việc, đồ án đã hoàn thành với các kết quả sau:

Sản phẩm hoàn thiện:

Game 2D hoạt động ổn định trên PC Windows

2 chế độ chơi: Single Player và Vs AI

Giao diện Neon Style hiện đại, thu hút

Âm thanh đầy đủ: BGM + SFX

Hỗ trợ 2 ngôn ngữ: Tiếng Việt và Tiếng Anh

Tính năng kỹ thuật:

Grid System với HashSet cho collision detection $O(1)$

AI với 2 chế độ: Hunting (BFS) và Survival (Flood Fill)

Input Buffering để tránh nuốt phím

Object Pooling cho audio system

Data Persistence với JSON

Localization system hoàn chỉnh

Hiệu năng:

FPS ổn định: 60 FPS

RAM usage: 150-200MB

CPU usage: < 10% trên Core i5-12500H

Load time: < 2 giây

4.2 Kịch bản kiểm thử (Test Cases)

Để đảm bảo chất lượng phần mềm, quá trình kiểm thử được thực hiện qua các kịch bản sau:

Bảng 2: Danh sách các mục kiểm thử

STT	Chức năng	Thao tác thực hiện	Kết quả mong đợi	Kết quả thực tế	Trạng thái
1	Di chuyển	Nhấn phím W, A, S, D	Rắn đổi hướng đúng theo phím nhấn, không đi ngược chiều.	Như mong đợi	PASS
2	Ăn mồi	Điều khiển rắn đâm vào mồi	Điểm tăng, rắn dài ra, mồi biến mất và xuất hiện chỗ khác.	Như mong đợi	PASS
3	Va chạm tường	Đâm vào biên	Game Over	Như mong đợi	PASS
4	Va chạm thân	Đâm vào thân	Game Over	Như mong đợi	PASS
5	AI tìm đường	Quan sát AI trong chế độ PvE	AI tự động đi đến mồi, tránh tường.	AI đôi khi bị kẹt nếu mồi sinh trong góc kín.	PASS(90%)
6	AI survival	AI bị bao vây	Tìm vùng rộng	Như mong đợi	PASS
7	Pause/Resume	Nhấn ESC	Game tạm dừng	Như mong đợi	PASS
8	Đổi ngôn ngữ	Vào Cài đặt Chọn Tiếng	Toàn bộ Text chuyển sang	Như mong đợi	PASS

		Anh	Tiếng Anh.		
9	Lưu điểm	Chơi đạt điểm cao -> Tắt game -> Mở lại	Bảng xếp hạng vẫn giữ nguyên tên và điểm cũ.	Như mong đợi	PASS
10	Âm thanh	Điều chỉnh	Volume thay đổi	Như mong đợi	PASS

4.3 Đánh giá hiệu năng (Performance)

FPS (Frames Per Second): Ổn định ở mức 60 FPS trên máy tính cấu hình văn phòng (Core i5, không card rời).

Bộ nhớ (RAM): Tiêu thụ khoảng 150-200MB. Việc sử dụng Object Pooling cho âm thanh giúp giảm thiểu việc cấp phát bộ nhớ liên tục (Garbage Collection), tránh hiện tượng giật lag.

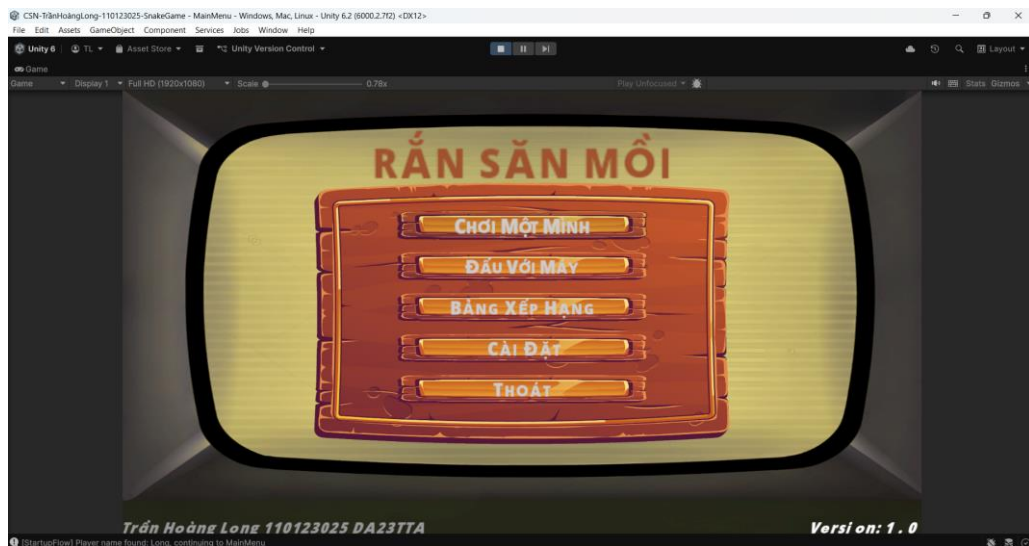
Tốc độ thuật toán: Thuật toán BFS tìm đường cho AI tốn trung bình 0.5ms - 1ms mỗi lần tính toán, hoàn toàn không ảnh hưởng đến trải nghiệm người chơi.

4.4 Giao diện và trải nghiệm người dùng

Giao diện được thiết kế theo phong cách Neon (Phát sáng) tạo cảm giác hiện đại.

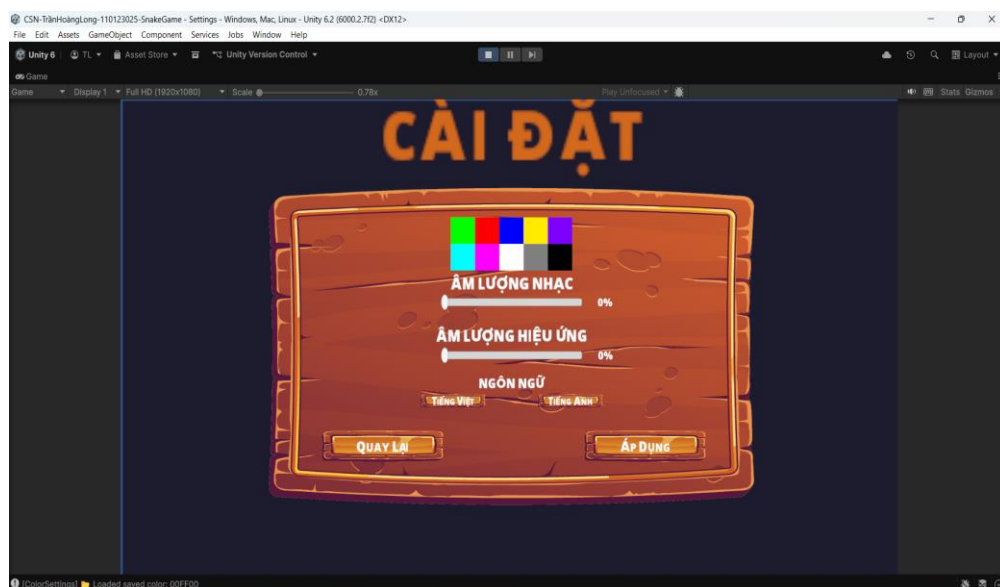
Hiệu ứng GlowEffect và OutlineEffect giúp phân biệt rõ ràng giữa rắn người chơi (Xanh lá) và rắn máy (Xanh dương).

Âm thanh phản hồi (Sound Feedback) đầy đủ khi di chuyển, ăn mồi, và Game Over.



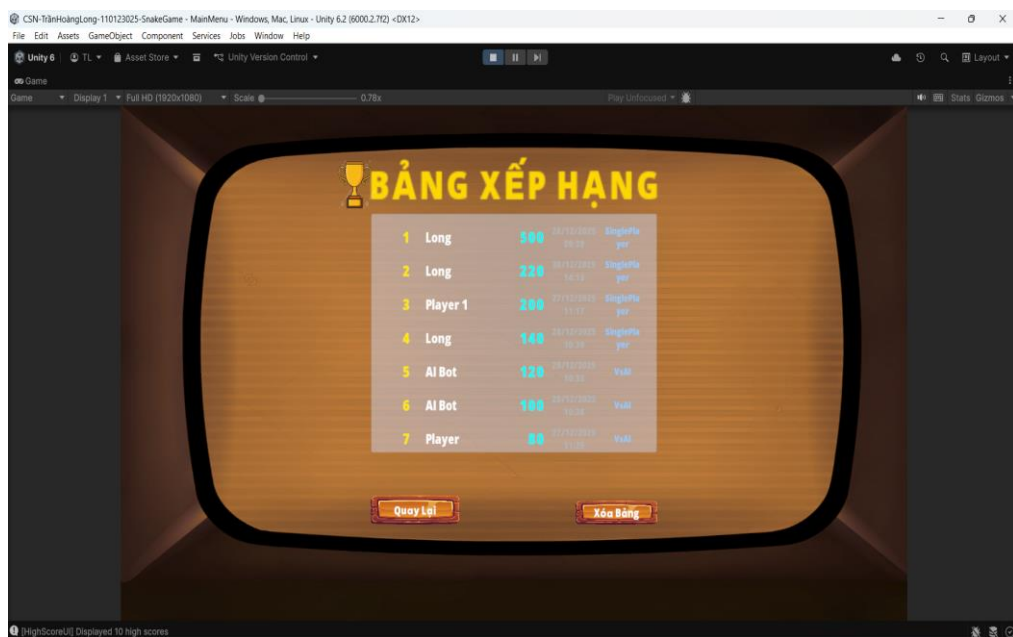
Hình 6: Giao diện Menu Chính

Giao diện Menu chính được thiết kế theo phong cách tối giản (Minimalism) kết hợp với tông màu Neon hiện đại trên nền xanh đen (#0F172A), tạo cảm giác huyền bí và tập trung thị giác cho người chơi. Các nút chức năng quan trọng như 'Chơi Ngay', 'Cài Đặt', 'Bảng Xếp Hạng' được bố trí chính giữa màn hình với kích thước lớn, dễ thao tác. Đặc biệt, hệ thống UI áp dụng script `AnimatedButton.cs` để tạo hiệu ứng phản hồi vật lý: khi người dùng rê chuột (Hover), nút sẽ tự động phóng to và phát sáng, kèm theo âm thanh 'bíp' nhẹ, giúp tăng trải nghiệm tương tác (UX). Logo trò chơi được đặt trang trọng ở phía trên cùng với hiệu ứng đổ bóng và phát sáng (Glow), khẳng định nhận diện thương hiệu của sản phẩm ngay từ cái nhìn đầu tiên.



Hình 7: Giao diện cài đặt

Màn hình Cài đặt không chỉ cho phép điều chỉnh các thông số kỹ thuật cơ bản như Âm lượng nhạc nền (BGM) hay Hiệu ứng âm thanh (SFX) mà còn là nơi người chơi thể hiện cá tính. Điểm nhấn của giao diện này là bảng màu 'Snake Skin' với 10 tùy chọn màu sắc rực rỡ. Khi người chơi chọn một màu, hệ thống lập tức lưu lại lựa chọn này vào PlayerPrefs và cập nhật hình ảnh xem trước (Preview) theo thời gian thực mà không cần tải lại game. Ngoài ra, tính năng chuyển đổi ngôn ngữ (Tiếng Việt / Tiếng Anh) cũng được tích hợp tại đây; nhờ cơ chế LocalizationManager, toàn bộ văn bản trên giao diện sẽ chuyển ngữ ngay lập tức, đảm bảo tính thuận tiện tối đa cho người sử dụng mà không làm gián đoạn trải nghiệm.



Hình 8: Giao diện bảng xếp hạng

Màn hình Kết thúc game được thiết kế để tổng kết lại toàn bộ quá trình chơi một cách trực quan nhất. Thông báo 'CHIẾN THẮNG' hoặc 'THẤT BẠI' được hiển thị khổ lớn với màu sắc tương ứng (Xanh hoặc Đỏ) để gây ấn tượng mạnh về cảm xúc. Ngay bên dưới là bảng tổng kết điểm số chi tiết, bao gồm điểm gốc và điểm thưởng từ các chuỗi Combo. Nếu người chơi phá kỷ lục cũ, một hiệu ứng pháo hoa (Particle System) sẽ được kích hoạt cùng dòng chữ 'KỶ LỤC MỚI!' nhấp nháy liên tục. Tại đây, người chơi có hai lựa chọn điều hướng rõ ràng: nút 'Chơi Lại' để bắt đầu ngay một ván mới với các cài đặt cũ, hoặc nút 'Menu' để quay về màn hình chính, hoàn thiện một vòng lặp trải nghiệm người dùng (User Flow) khép kín và mượt mà.



Hình 9: Màn hình Gameplay

Giao diện khi chơi (Ingame UI) được tối ưu hóa để không che khuất tầm nhìn nhưng vẫn cung cấp đầy đủ thông tin chiến thuật. Thanh trạng thái (HUD) phía trên cùng hiển thị điểm số của Người chơi (Player 1) và Máy (AI) ở hai góc đối diện, giúp người chơi dễ dàng so sánh cục diện trận đấu. Ở chính giữa là đồng hồ đếm ngược (Timer) tạo áp lực thời gian thực. Đặc biệt, hệ thống hiển thị 'Combo' sẽ xuất hiện nổi bật với hiệu ứng rung lắc (Shake) mỗi khi người chơi ăn mồi liên tiếp, kích thích sự hưng phấn. Các đối tượng trên bàn cờ như Rắn và Thức ăn đều được phủ một lớp GlowEffect nhẹ và viền Outline, giúp chúng tách biệt rõ ràng khỏi nền lưới (Grid), đảm bảo người chơi không bị rối mắt khi di chuyển ở tốc độ cao.



Hình 10: Màn hình Game Over

Màn hình Kết thúc game được thiết kế để tổng kết lại toàn bộ quá trình chơi một cách trực quan nhất. Thông báo 'CHIẾN THẮNG' hoặc 'THẤT BẠI' được hiển thị khổ lớn với màu sắc tương ứng (Xanh hoặc Đỏ) để gây ấn tượng mạnh về cảm xúc.

Ngay bên dưới là bảng tổng kết điểm số chi tiết, bao gồm điểm gốc và điểm thưởng từ các chuỗi Combo. Nếu người chơi phá kỷ lục cũ, một hiệu ứng pháo hoa (Particle System) sẽ được kích hoạt cùng dòng chữ 'KỶ LỤC MỚI!' nhấp nháy liên tục. Tại đây, người chơi có hai lựa chọn điều hướng rõ ràng: nút 'Chơi Lại' để bắt đầu ngay một ván mới với các cài đặt cũ, hoặc nút 'Menu' để quay về màn hình chính, hoàn thiện một vòng lặp trải nghiệm người dùng (User Flow) khép kín và mượt mà.

CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1 Kết luận

Đồ án đã giải quyết trọn vẹn bài toán đặt ra ban đầu. Không chỉ tái hiện thành công gameplay cổ điển, sản phẩm còn tích hợp được những công nghệ tiên tiến:

Hệ thống Grid System: Giúp game hoạt động chính xác, loại bỏ lỗi vật lý.

AI Thông minh: Tạo ra thử thách thực sự cho người chơi thay vì những con bot vô tri.

Kiến trúc phần mềm chuẩn: Code được tổ chức khoa học, dễ dàng mở rộng và bảo trì.

Qua quá trình thực hiện, sinh viên đã rút ra được nhiều bài học quý giá về:

Tư duy lập trình thuật toán trong môi trường thời gian thực (Real-time).

Cách quản lý tài nguyên và tối ưu hóa hiệu năng trong Unity.

Kỹ năng Debug và xử lý các lỗi logic phức tạp.

5.2 Hạn chế

Tuy nhiên, sản phẩm vẫn còn một số điểm cần khắc phục:

Thuật toán AI đôi khi vẫn chưa tối ưu trong việc "bẫy" người chơi.

Chưa có chế độ chơi qua mạng (Online Multiplayer).

Hiệu ứng hình ảnh còn khá đơn giản, chưa có nhiều Skin đẹp mắt.

5.3 Hướng phát triển trong tương lai

Tích hợp Multiplayer: Sử dụng thư viện Photon PUN hoặc Mirror để cho phép 2 người chơi kết nối qua Internet.

Nâng cấp AI với Machine Learning: Sử dụng Unity ML-Agents để huấn luyện AI bằng phương pháp Học tăng cường (Reinforcement Learning), giúp AI tự học chiến thuật mới thay vì code cứng (Hard-code).

Phát hành Mobile: Tối ưu hóa hệ thống điều khiển cảm ứng (Touch Controls) để đưa game lên kho ứng dụng Google Play và App Store.

DANH MỤC TÀI LIỆU THAM KHẢO

- [1]. Lê Minh Hoàng, Giải thuật và Lập trình. NXB Đại học Sư phạm Hà Nội, 2002.
- [2]. Nguyễn Đức Nghĩa, Giáo trình Cấu trúc dữ liệu và giải thuật. NXB Đại học Bách Khoa Hà Nội, 2005.
- [3]. Phạm Hữu Tình, Giáo trình Lập trình hướng đối tượng với C#. NXB Xây Dựng, 2012.
- [4]. <https://howkteam.vn/learn/lap-trinh/unity3d-7-20>. Các khoá học về lập trình Unity 2D .
- [5]. J. Hocking, Unity in Action, 2nd ed. Manning Publications, 2018.
- [6]. R. Nystrom, Game Programming Patterns. Genever Benning, 2014.
- [7]. Unity Technologies. (2023). Unity User Manual 2023.1.
- [8]. Microsoft Documentation. C# Programming Guide.
- [9]. GeeksforGeeks. Breadth First Search or BFS for a Graph.